

Trabalho de Pesquisa e Aplicação Prática
Engenharia e Design de Interfaces Analíticas na Indústria 4.0

PredictaMaq

Manutenção Preditiva e Telemetria de Maquinário Crítico
(Prensas, Compressores e Tornos CNC)

Relatório Técnico

Equipe 2

[Inserir nomes dos integrantes]

[Inserir curso, turma e docente]

1 de outubro de 2026

Sumário

Resumo	3
1 Introdução	4
1.1 Contexto e objetivo	4
1.2 Escopo	4
2 Resposta direta: qual framework de código foi utilizado?	4
3 Fundamentação teórica	5
3.1 Fundamentos de UX para visualização de dados	5
3.2 Manutenção preditiva e monitoramento de condição	6
3.3 Frameworks de código para visualização	7
3.4 Integração web e tempo real	7
3.5 Ferramentas de BI	7
4 Metodologia e processo de desenvolvimento	7
5 Arquitetura e implementação	8
5.1 Visão geral	8
5.2 Dados (“banco” em CSV)	9
5.3 Limites operacionais (fictícios)	10
5.4 Simulador de telemetria	10
5.5 Análises	10
5.6 Persistência	11
6 Integração web e tempo real	11
6.1 WebSocket e API REST	11
6.2 Fluxo no navegador	12
6.3 Responsividade	12
7 Design da interface	12
7.1 Da primeira versão ao redesenho	12
7.2 Telas	14
7.3 Cores e contraste	21
8 Testes e verificação	21
8.1 Testes unitários	21
8.2 Testes ponta a ponta	21
8.3 Defeitos encontrados e corrigidos	22
8.4 Medições de desempenho	22

9 Diagnóstico crítico	22
9.1 Falhas de usabilidade	23
9.2 Gargalos de performance e arquitetura	23
9.3 Oportunidades de melhoria	23
10 Atendimento à rubrica	24
11 Conclusão	24
A Como executar	24
B Roteiro sugerido para a demonstração (15 minutos)	25

Resumo

Este relatório documenta o desenvolvimento do *PredictaMaq*, uma aplicação web que simula o painel de um engenheiro de manutenção e confiabilidade monitorando prensas hidráulicas, compressores de parafuso e tornos CNC por meio de telemetria de vibração, temperatura e pressão. O sistema gera dados fictícios, armazena-os em arquivos CSV, transmite leituras em tempo real por WebSocket, calcula um índice de saúde, detecta anomalias, estima a vida útil restante (RUL) e gerencia alertas e ordens de serviço. O **framework de código** utilizado é o **Dash (Plotly)**, sobre Flask, conforme a stack recomendada para a Equipe 2 na proposta do trabalho. O relatório descreve a fundamentação teórica, as decisões de arquitetura, o processo de desenvolvimento (incluindo um redesenho completo da interface após avaliação crítica), os testes, as limitações do protótipo e o mapeamento às capacidades avaliadas na rubrica.

Palavras-chave: manutenção preditiva; telemetria; visualização de dados; Dash; Plotly; WebSocket; usabilidade.

1 Introdução

1.1 Contexto e objetivo

A proposta de trabalho pede a construção de interfaces analíticas para o ambiente fabril da Indústria 4.0, unindo fundamentos de design visual, desenvolvimento web, análise de dados e ferramentas de *Business Intelligence*. Cada equipe recebeu um tema específico; o da Equipe 2 é **Manutenção Preditiva e Telemetria de Maquinário Crítico (prensas/compressores)**. O enunciado detalha que o painel deve processar séries temporais de vibração, temperatura e pressão e emitir alertas visuais imediatos em caso de desvio dos limites operacionais, tendo como público-alvo *engenheiros de manutenção e equipe de confiabilidade*.

O objetivo do projeto foi construir uma aplicação web operacional, com dados fictícios persistidos em CSV, que demonstrasse ao mesmo tempo o tema específico (manutenção preditiva) e o tema geral exigido de todas as equipes: fundamentos de UX, frameworks de código, integração web e ferramentas de BI.

1.2 Escopo

- Seis máquinas: duas prensas hidráulicas (PR-01, PR-02), dois compressores de parafuso (CP-01, CP-02) e dois tornos CNC (CNC-01, CNC-02).
- Três variáveis por máquina: vibração (mm/s), temperatura (°C) e pressão (bar).
- Histórico de 14 dias (leituras a cada 10 minutos) e telemetria ao vivo (uma leitura a cada 2 segundos).
- Análise: índice de saúde, anomalias, tendência e vida útil restante (RUL).
- Fluxo de manutenção: alerta → reconhecimento → ordem de serviço → conclusão.

2 Resposta direta: qual framework de código foi utilizado?

O framework principal é o **Dash**, da Plotly (versão 4.4.1 no ambiente de desenvolvimento), escrito em **Python**. O Dash executa sobre o servidor **Flask** e renderiza os gráficos com a biblioteca **Plotly** (Plotly.js no navegador). A escolha segue a stack recomendada no quadro de equipes da proposta para o tema da Equipe 2: *Python (Plotly/Dash) + WebSockets/Streaming + API de Sensores*.

Tabela 1: Tecnologias utilizadas e seus papéis.

Camada	Tecnologia	Papel no projeto
Framework de interface	Dash 4.4 (Plotly)	Layout, roteamento, callbacks reativos e componentes (<code>dcc</code> , <code>html</code>).
Gráficos	Plotly 7.1 (Plotly.js)	Séries temporais, projeção de tendência, barras, zonas de limite.
Servidor web	Flask 3.1	Serve a aplicação Dash e as rotas REST (<code>/api/*</code>).
Tempo real	Flask-Sock 0.7	Endpoint WebSocket <code>/ws</code> que envia cada leitura aos navegadores.
Dados e análise	pandas 3.0, NumPy 2.4	Leitura de CSV, reamostragem, regressão, estatísticas robustas.
Persistência	Arquivos CSV	“Banco” de histórico, telemetria ao vivo, alertas e ordens de serviço.
Front-end próprio	HTML5, CSS3, JavaScript	Tema claro/escuro, layout responsivo e cliente WebSocket (<code>assets/</code>).
Qualidade (desenvolvimento)	pytest, Playwright	Testes unitários da análise e testes ponta a ponta no navegador.

Não foram utilizados D3.js, Chart.js, Streamlit nem ferramentas de BI de mercado (Power BI, Tableau) na implementação; esses itens aparecem na comparação teórica da Seção 3. As versões acima são as instaladas durante o desenvolvimento; o arquivo `requirements.txt` fixa apenas versões mínimas.

3 Fundamentação teórica

Nota: as referências citadas nesta seção e na bibliografia são pontos de partida levantados durante o projeto. Edição, ano e conteúdo devem ser conferidos nas fontes originais antes da entrega final.

3.1 Fundamentos de UX para visualização de dados

A proposta cita explicitamente as Leis da Gestalt, a hierarquia visual, o contraste, as paletas de cores acessíveis e a gestão da carga cognitiva. Esses conceitos orientaram as decisões de interface:

Proximidade e região comum.

Elementos relacionados devem ficar próximos e, se possível, contidos numa mesma região. No projeto, cada máquina é um cartão; o espaço entre seções é maior que o espaço dentro delas.

Similaridade.

Elementos com a mesma função têm a mesma aparência. O estado de uma máquina é sempre exibido por um mesmo componente (“chip”), com a mesma lógica de cor e forma em todas as telas.

Continuidade e alinhamento.

A lista de máquinas usa colunas alinhadas, permitindo comparar valores por varredura vertical.

Figura-fundo.

Nos gráficos, a série ocupa o primeiro plano e as zonas de atenção e crítico aparecem como fundo translúcido.

Hierarquia visual.

Cada tela tem um elemento dominante (por exemplo, a frase de estado da planta ou os dias restantes até o limite crítico), seguido de informação secundária e, por último, detalhes.

Carga cognitiva e revelação progressiva.

Uma pergunta por tela, uma variável por gráfico, e detalhes técnicos escondidos em blocos recolhíveis.

Cor acessível.

O estado nunca depende só da cor: usa-se cor, forma (círculo, triângulo, quadrado) e texto. Os contrastes foram calculados (Seção 7.3), seguindo os critérios de contraste da WCAG 2.x.

3.2 Manutenção preditiva e monitoramento de condição

A manutenção pode ser corretiva (após a falha), preventiva (por calendário) ou preditiva (baseada na condição real do ativo). O monitoramento de condição mede grandezas como vibração, temperatura e pressão e as compara com zonas de severidade (normal, atenção, crítico). A estimativa de vida útil restante (*Remaining Useful Life*, RUL) projeta a evolução de um indicador de degradação até um limite de falha. Normas de referência na área incluem a ISO 20816 (vibração em máquinas), a ISO 17359 (diretrizes de monitoramento de condição) e a ISO 13374 (processamento e apresentação de dados). **Neste projeto, os limites numéricos são fictícios e didáticos**, não derivados de norma ou fabricante.

3.3 Frameworks de código para visualização

Tabela 2: Comparativo de tecnologias de visualização considerado pela equipe.

Opção	Pontos fortes	Limitações	Quando usar
Dash + Plotly (usado)	Tudo em Python; callbacks reativos; gráficos interativos prontos; integra com pandas.	Cada interação passa pelo servidor; menos controle fino do visual.	Dashboards analíticos de times de dados e engenharia.
D3.js	Controle total; alto desempenho.	Curva de aprendizado alta; muito código.	Visualizações customizadas.
Chart.js	Leve e simples para gráficos padrão.	Menos recursos analíticos nativos.	Painéis simples embutidos em páginas.
Streamlit	Prototipagem muito rápida.	Reexecuta o script a cada interação; layout menos flexível.	Exploração e protótipos internos.
Power BI / Tableau	Autosserviço, governança, conectores corporativos.	Tempo real limitado; licenças; menos customização.	BI corporativo e relatórios gerenciais.

3.4 Integração web e tempo real

Há duas abordagens principais para atualizar um painel: *polling* (o navegador pergunta periodicamente) e *push* (o servidor envia quando há dado novo). O protocolo WebSocket (RFC 6455) mantém uma conexão bidirecional aberta, reduzindo latência e tráfego. O projeto usa WebSocket para anunciar cada leitura nova e mantém uma API REST para consulta pontual (Seção 6).

3.5 Ferramentas de BI

Power BI e Tableau permitem construir painéis a partir de CSV, SQL e outras fontes com pouco código, e oferecem governança, compartilhamento e conectores corporativos. Em contrapartida, a atualização em tempo real e a personalização visual são mais limitadas do que numa aplicação própria. Os CSVs gerados por este projeto (**data/**) podem ser carregados diretamente em ferramentas de BI para comparação; essa comparação prática **não foi realizada** neste trabalho e fica como sugestão.

4 Metodologia e processo de desenvolvimento

O trabalho foi conduzido de forma iterativa, com avaliação crítica a cada ciclo.

1. **Análise da proposta.** Leitura do enunciado, identificação do público-alvo, da stack recomendada e da rubrica de avaliação.

2. **Escolha da solução.** Foi proposta uma aplicação web com dados fictícios em CSV. Ajustou-se o escopo: em vez de um ERP completo, um painel de monitoramento de condição com um módulo enxuto de ordens de serviço. Para o tempo real, um simulador de telemetria transmite leituras novas, enquanto o CSV guarda histórico, alertas e ordens.
3. **Primeira versão (v1).** Backend, simulador, análises e uma interface com cinco páginas, barra lateral escura, KPIs, cartões com minigráficos, mapa de calor, três gráficos empilhados por máquina, tabelas e uma página de diagnóstico.
4. **Avaliação crítica.** Ao revisar a v1, a equipe concluiu que havia informação demais e pouca hierarquia, e que os princípios visuais citados no enunciado não estavam aplicados com rigor.
5. **Redesenho (v2).** Reestruturação da interface com uma pergunta por tela, revelação progressiva, componentes consistentes, tema claro e escuro com contrastes calculados e layout responsivo (Seção 7).
6. **Verificação.** Testes unitários da lógica de análise e testes ponta a ponta no navegador, com captura de telas (Seção 8).

5 Arquitetura e implementação

5.1 Visão geral

```
simulador (thread) --> CSV (historico, ao vivo, alertas, ordens)
|
+--> Flask-Sock /ws --> navegador (assets/live.js) --> dcc.Store --> callbacks
Dash/Plotly
API REST: /api/sensors /api/history?machine=PR-01 /api/snapshot /api/health
```

Tabela 3: Organização do código-fonte.

Arquivo	Responsabilidade
<code>run.py</code>	Ponto de entrada: gera dados se necessário, inicia o simulador e o servidor.
<code>predictive/config.py</code>	Máquinas, variáveis, limites operacionais, perfis de desgaste, falhas e incidentes.
<code>predictive/simulator_model.py</code>	Modelo dinâmico de cada máquina (carga, desgaste, falhas, ruído).
<code>predictive/datagen.py</code>	Geração do histórico de 14 dias, alertas e ordens iniciais.
<code>predictive/analytics.py</code>	Severidade, índice de saúde, anomalias, RUL e detector de alertas.
<code>predictive/stream.py</code>	Simulador em tempo real, hub WebSocket e API REST.
<code>predictive/store.py</code>	Persistência em CSV com lock e cache.
<code>predictive/insights.py</code>	Textos em linguagem natural (motivo do estado, tendência, recomendação).
<code>predictive/figs.py, ui.py, pages.py, app.py</code>	Figuras, componentes, páginas e callbacks.
<code>assets/style.css, live.js, early_theme.js</code>	Estilo e tema, cliente WebSocket e aplicação antecipada do tema.
<code>tests/test_analytics.py</code>	Quatro testes unitários da lógica de análise.

5.2 Dados (“banco” em CSV)

Os arquivos ficam em `data/` e são gerados na primeira execução (e regenerados se o histórico tiver mais de 12 horas).

Tabela 4: Arquivos CSV e suas colunas principais.

Arquivo	Colunas
<code>telemetry_history.csv</code>	<code>timestamp</code> , <code>machine_id</code> , <code>vibration_mm_s</code> , <code>temperature_c</code> , <code>pressure_bar</code> , <code>load_pct</code> (12 096 linhas: 6 máquinas × 2016 leituras)
<code>telemetry_live.csv</code>	Mesmas colunas; recriado a cada execução e preenchido pelo simulador ao vivo.
<code>alerts.csv</code>	<code>alert_id</code> , <code>timestamp</code> , <code>machine_id</code> , <code>variable</code> , <code>severity</code> , <code>value</code> , <code>limit</code> , <code>status</code> , <code>message</code> , <code>acknowledged_by</code> , <code>work_order_id</code>
<code>work_orders.csv</code>	<code>wo_id</code> , <code>created_at</code> , <code>machine_id</code> , <code>alert_id</code> , <code>priority</code> , <code>type</code> , <code>description</code> , <code>status</code> , <code>assigned_to</code> , <code>due_date</code>
<code>machines.csv</code>	<code>machine_id</code> , <code>name</code> , <code>type</code> , <code>area</code>

5.3 Limites operacionais (fictícios)

Cada tipo de máquina tem um valor nominal (Nom., a 60% de carga) e limites de atenção e crítico. A pressão tem limites nos dois sentidos; as colunas com seta para baixo ($\downarrow$) são os limites inferiores.

Tabela 5: Limites utilizados (valores fictícios para fins didáticos).

Tipo	Variável	Nom.	Atenção	Crítico	Atenção $\downarrow$	Crítico $\downarrow$
Prensa	Vibração (mm/s)	2,5	4,5	7,1	—	—
	Temperatura (°C)	55	70	85	—	—
	Pressão (bar)	180	200	215	165	150
Compressor	Vibração (mm/s)	2,0	4,0	6,3	—	—
	Temperatura (°C)	85	100	110	—	—
	Pressão (bar)	8,0	9,0	10,0	7,0	6,0
Torno CNC	Vibração (mm/s)	1,2	2,8	4,5	—	—
	Temperatura (°C)	40	55	65	—	—
	Pressão (bar)	6,0	7,5	8,5	4,5	3,5

5.4 Simulador de telemetria

Cada variável de cada máquina segue um alvo com dinâmica de primeira ordem. Para um intervalo Δt e constante de tempo τ :

$$x_{t+\Delta t} = x_t + \alpha (x_t^* - x_t), \quad \alpha = 1 - e^{-\Delta t/\tau}, \quad (1)$$

em que o alvo depende da carga c (%), do desgaste w e de eventuais falhas f :

$$x^* = x_{\text{nom}} \left(1 + k \frac{c-60}{100} \right) + w \Delta_{\text{crit}} + f \Delta_{\text{crit}}, \quad (2)$$

sendo Δ_{crit} a distância entre o valor nominal e o limite crítico, e k a sensibilidade à carga. A leitura registrada é o estado somado a um ruído gaussiano de medição. A carga segue um perfil diário com dois turnos produtivos, madrugada ociosa e menor atividade nos fins de semana.

No histórico, três máquinas têm degradação programada: a PR-02 com vibração crescente (forma exponencial), o CP-01 com temperatura crescente e queda de pressão, e o CNC-02 com vibração moderada e picos intermitentes. Quatro incidentes passados (vazamento, superaquecimento, falha de rolamento) foram inseridos para gerar alertas históricos. Para demonstração, a interface permite **injetar falhas** (rolamento, superaquecimento, vazamento), que crescem gradualmente em cerca de 20 segundos.

5.5 Análises

Índice de saúde. Para cada variável, define-se a pontuação de desvio $s_i \in [0, 1]$ como a fração do caminho entre o valor nominal e o limite crítico (no sentido do desvio). O índice

combina o pior desvio e a média ponderada (pesos $w_{\text{vib}} = 0,5$, $w_{\text{temp}} = 0,3$, $w_{\text{pres}} = 0,2$):

$$\text{IS} = 100 \left(1 - \min \left(1, 0,5 \max_i s_i + 0,5 \sum_i w_i s_i \right) \right). \quad (3)$$

O *estado* (normal, atenção ou crítico) é determinado diretamente pelos limites, independentemente do índice; o índice é uma métrica complementar. Na interface, as faixas do índice são: atenção abaixo de 75 e crítico abaixo de 45.

Anomalias. Usa-se o z-score robusto com mediana e desvio absoluto mediano (MAD) numa janela móvel de 72 leituras:

$$z = \frac{0,6745 (x - \text{mediana})}{\text{MAD}}, \quad (4)$$

marcando como anomalia os pontos com $|z| > 3,5$.

Vida útil restante (RUL). Ajusta-se uma reta (mínimos quadrados) às médias horárias dos últimos 5 dias e extrapola-se até o limite crítico L :

$$\text{RUL} = \frac{L - \hat{y}(t_{\text{agora}})}{b}, \quad (5)$$

em que b é a inclinação. Inclinações desprezíveis (menos de 0,4% da distância nominal-crítico por dia) são tratadas como “estável”. O coeficiente de determinação R^2 é exibido como indicador de confiança (alta $\geq 0,8$; média $\geq 0,5$; baixa abaixo disso). Estimativas acima de 60 dias são apresentadas como estáveis. **Trata-se de um modelo didático**, que assume tendência linear.

Detector de alertas. Um alerta é aberto quando o nível de severidade piora e se mantém por 3 leituras consecutivas (*debounce*, para filtrar ruído). O nível só é liberado após 8 leituras consecutivas melhores (histerese). Alertas pendentes do mesmo problema (máquina, variável e severidade) não são duplicados.

5.6 Persistência

As escritas em CSV são serializadas por um *lock* de processo, e a leitura do histórico é cacheada e recarregada somente quando o arquivo muda. Alertas e ordens de serviço são regravados integralmente a cada alteração, solução adequada ao porte do protótipo, mas não escalável (Seção 9).

6 Integração web e tempo real

6.1 WebSocket e API REST

O servidor expõe os seguintes pontos de acesso, além das rotas do próprio Dash:

Tabela 6: Interfaces de integração.

Ponto de acesso	Tipo	Função
/ws	WebSocket	Envia a cada tick (2 s) as leituras de todas as máquinas, novos alertas e falhas ativas.
/api/sensors	GET	Últimas leituras de cada máquina (API de sensores).
/api/history?machine=PR-01	GET	Histórico em JSON (máximo de 5 000 linhas).
/api/snapshot	GET	Buffer recente de todas as máquinas.
/api/health	GET	Estado do serviço (clientes WebSocket e contagem de ticks).

6.2 Fluxo no navegador

O script `assets/live.js` abre o WebSocket, reconecta com recuo exponencial em caso de queda e publica um pequeno marcador (contador, horário e quantidade de alertas) num componente `dcc.Store`. Os *callbacks* do Dash reagem a esse marcador, leem os dados do buffer do servidor e redesenham as figuras. Um indicador no cabeçalho mostra o estado da conexão (“Tempo real”, “Conectando...” ou “Reconectando...”). Nesta versão, portanto, o WebSocket funciona como *notificação push*, e os gráficos são redesenhados no servidor, uma limitação discutida na Seção 9.

6.3 Responsividade

O layout usa CSS Grid e Flexbox, com ajustes para telas estreitas (Figura 9). A barra de navegação passa para uma segunda linha rolável, e as listas se reorganizam em uma coluna.

7 Design da interface

7.1 Da primeira versão ao redesenho

A v1 reunia, numa mesma tela, KPIs, cartões com minigráficos, mapa de calor, três gráficos empilhados, tabelas e explicações, e exigia que o usuário decidisse sozinho o que importava. O redesenho partiu de uma pergunta por tela:

Tabela 7: Estrutura da interface após o redesenho.

Rota	Pergunta	Conteúdo
/	O que precisa da minha atenção agora?	Frase de estado da planta, cartões das máquinas problemáticas (motivo e tendência) e lista compacta de todas.
/monitoramento	Como está esta máquina agora?	Três variáveis como abas-resumo; um gráfico em tempo real por vez com zonas de limite; simulador de falhas recolhido.
/analise	Quando preciso agir?	Diagnóstico em linguagem natural, dias até o limite crítico, tendência, índice de saúde, comparação; ordem de serviço em um clique.
/alertas	O que preciso tratar?	Alertas e ordens com a próxima ação em cada linha; exportação CSV.
/sobre	Como foi feito?	Princípios de design, diagnóstico crítico, tecnologias, arquitetura e medições.

7.2 Telas

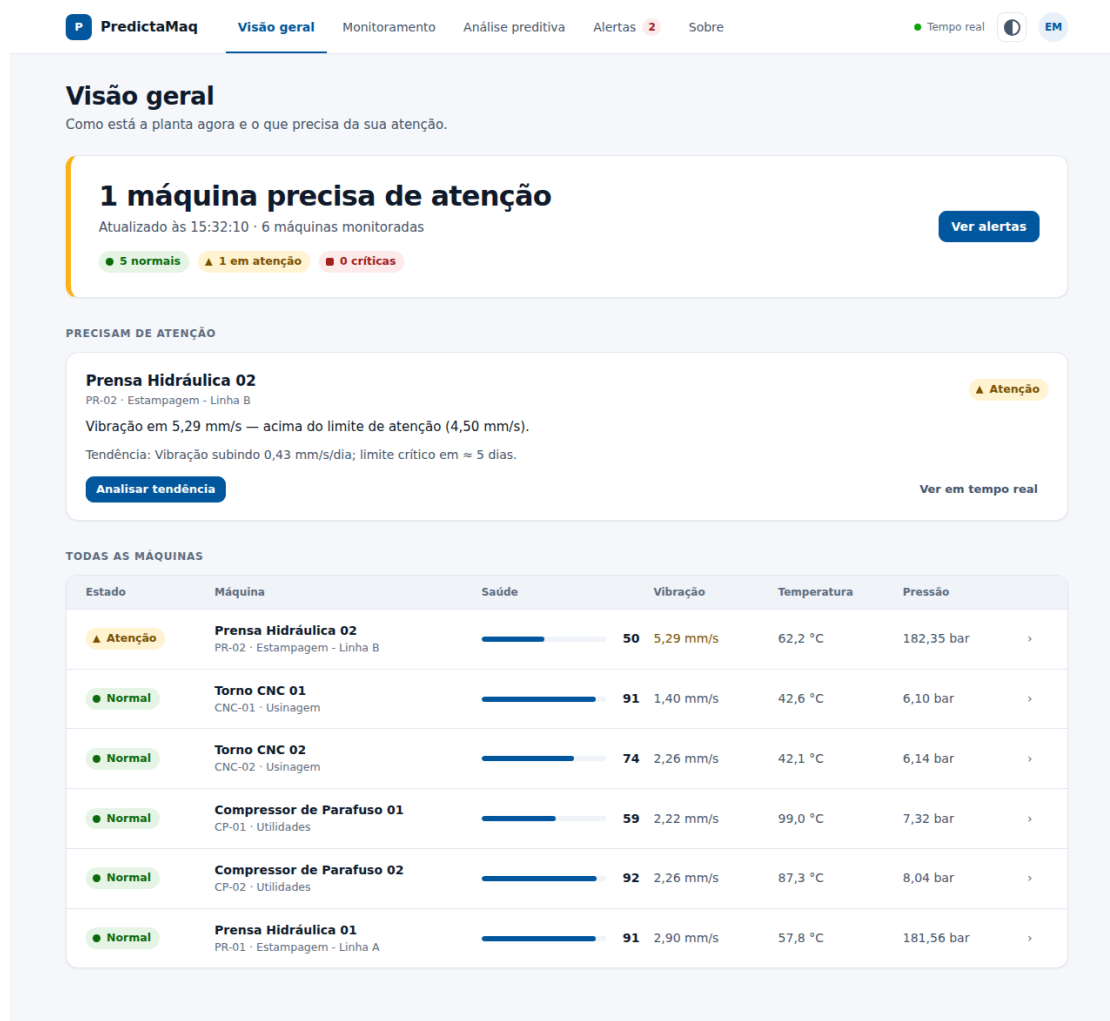


Figura 1: Visão geral com uma máquina em atenção (PR-02): o cartão explica o motivo e a tendência, e a lista abaixo mostra todas as máquinas.

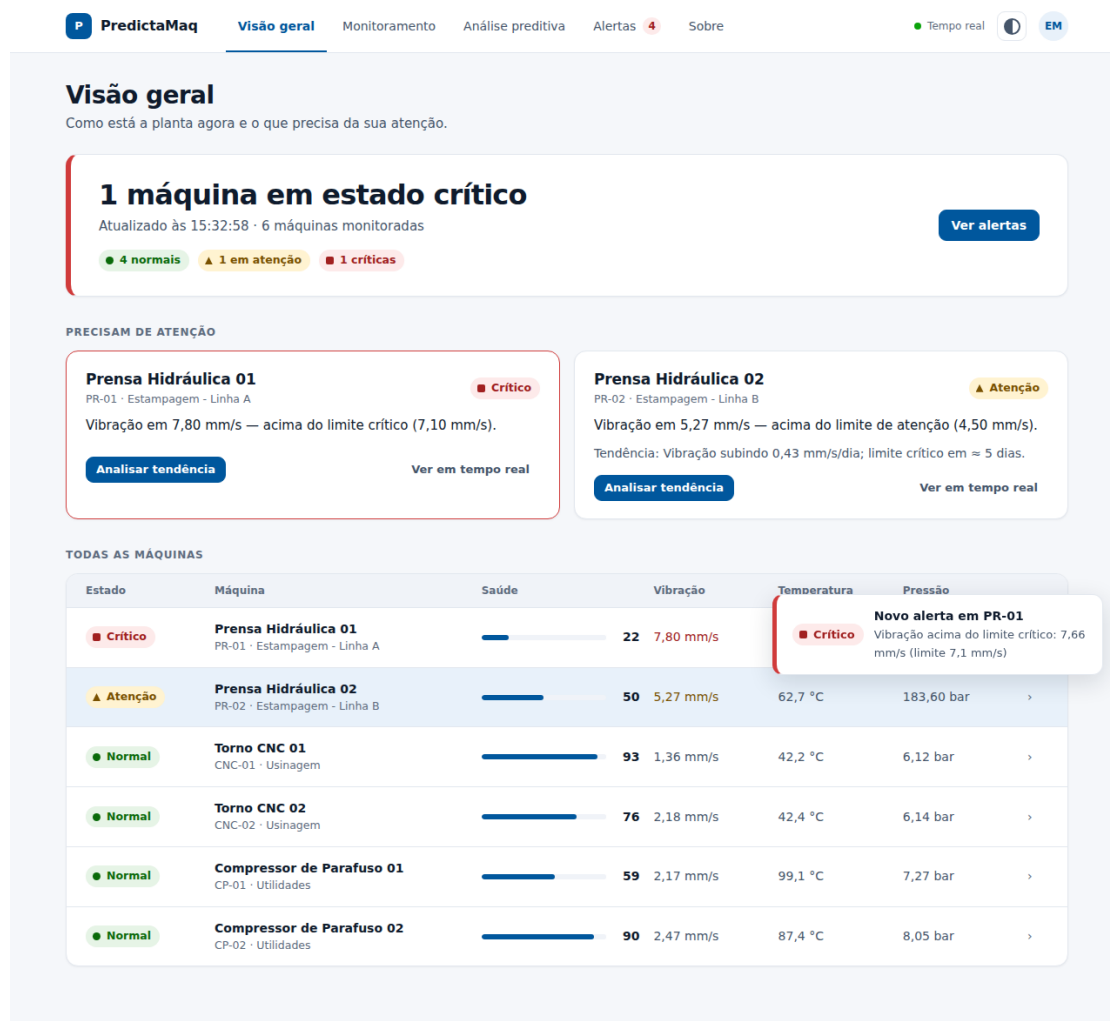


Figura 2: Visão geral após injetar uma falha: o estado crítico domina a hierarquia e os cartões priorizam a máquina afetada.

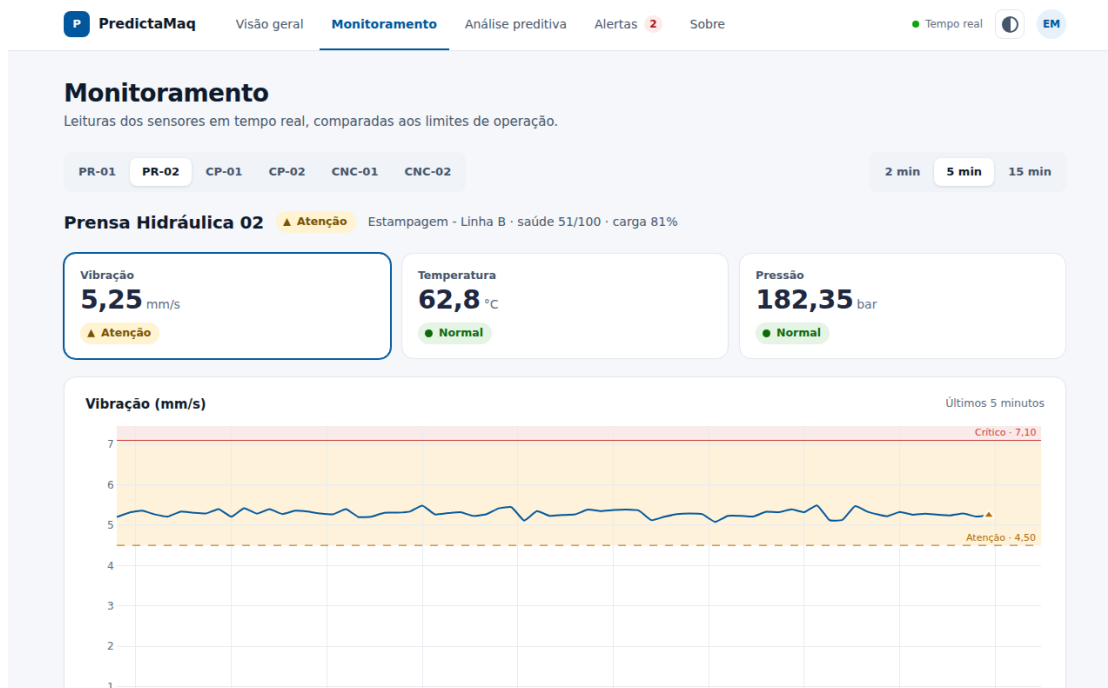


Figura 3: Monitoramento: as abas mostram o valor atual e o estado de cada variável; o gráfico mostra a variável selecionada com as zonas de atenção e crítico.

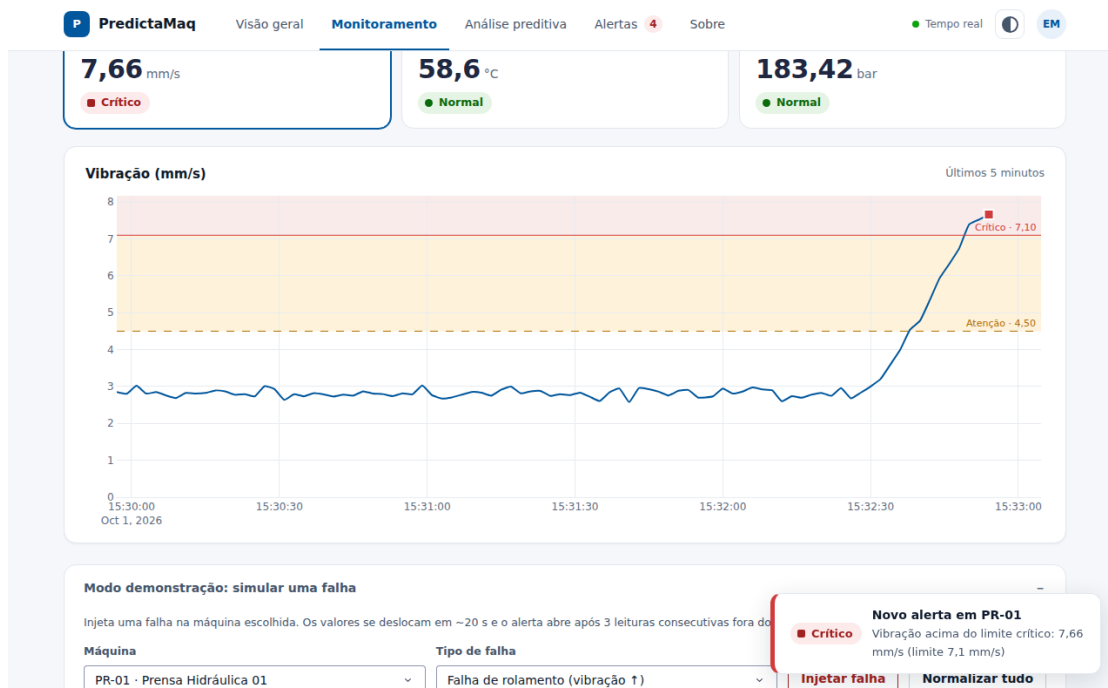


Figura 4: Monitoramento durante a falha de rolamento simulada na PR-01, com notificação de novo alerta no canto inferior direito.

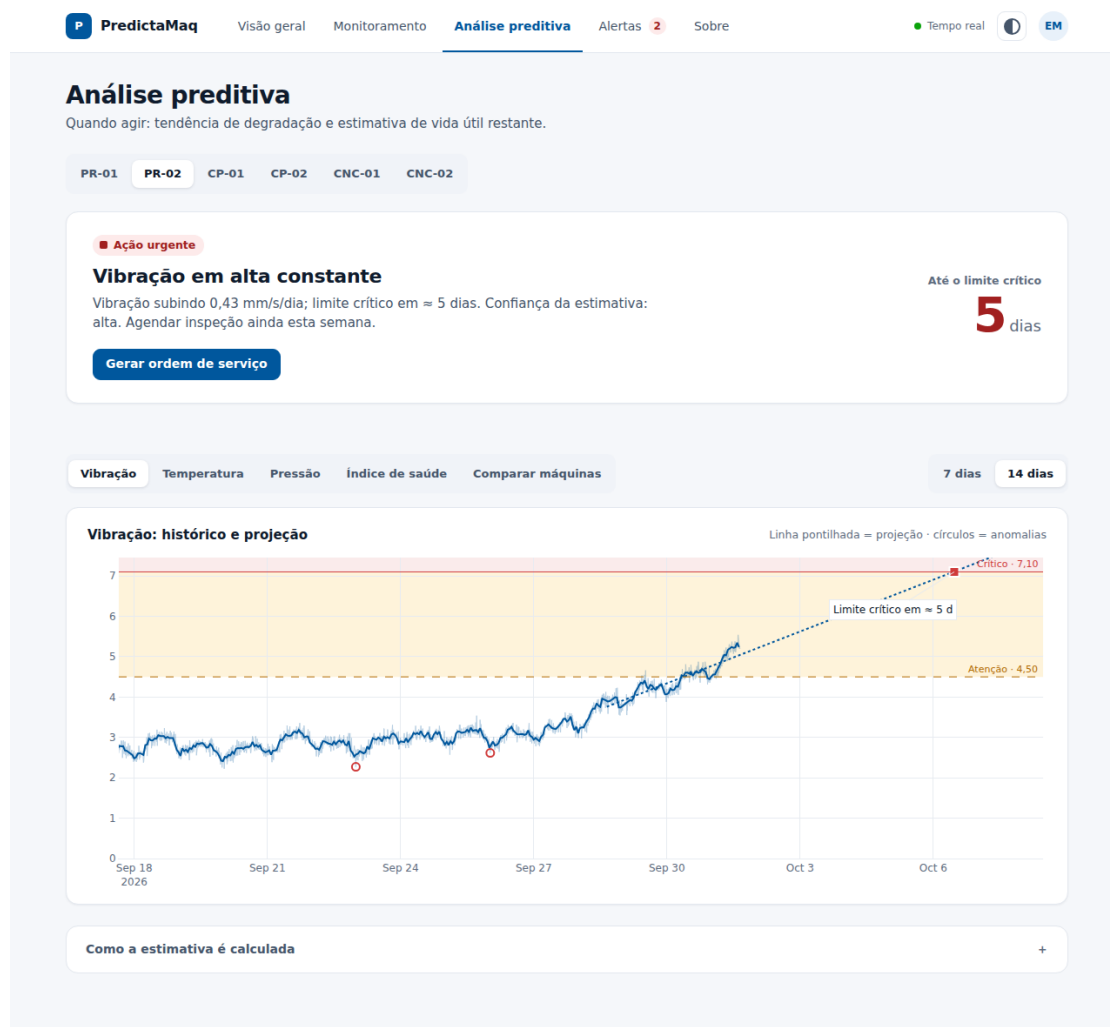


Figura 5: Análise preditiva da PR-02: diagnóstico em linguagem natural, dias até o limite crítico e projeção da tendência com marcação do cruzamento previsto.

The screenshot shows the 'Alertas e manutenção' (Alerts and maintenance) section of the PredictaMq interface. The top navigation bar includes 'PredictaMq', 'Visão geral', 'Monitoramento', 'Análise preditiva', 'Alertas 2', and 'Sobre'. On the right, there are indicators for 'Tempo real' (Real time) and 'EM' (Emergency). The main heading is 'Alertas e manutenção', with a subtext: 'Trate o que está pendente: reconheça o alerta, abra uma ordem de serviço e acompanhe até concluir.' and a link to 'Exportar CSV'. Below this, there are tabs for 'Alertas' and 'Ordens de serviço'. On the right, there are filters for 'Pendentes' (selected), 'Concluídos', and 'Todos'. The main content area displays two alerts, each with a yellow 'Atenção' (Attention) icon and a blue 'Reconhecer' (Recognize) button, followed by a grey 'Criar OS' (Create OS) button.

Alerta	Descrição	Ações
CP-01 · Compressor de Parafuso 01	Temperatura acima do limite de atenção: 100,9 °C (limite 100 °C) há 23 h · Aberto	Reconhecer, Criar OS
PR-02 · Prensa Hidráulica 02	Vibração acima do limite de atenção: 4,57 mm/s (limite 4,5 mm/s) há 1 d · Aberto	Reconhecer, Criar OS

At the bottom, there is a button labeled 'Nova ordem de serviço manual' with a '+' icon.

Figura 6: Alertas pendentes, cada um com a ação seguinte mais provável em destaque.

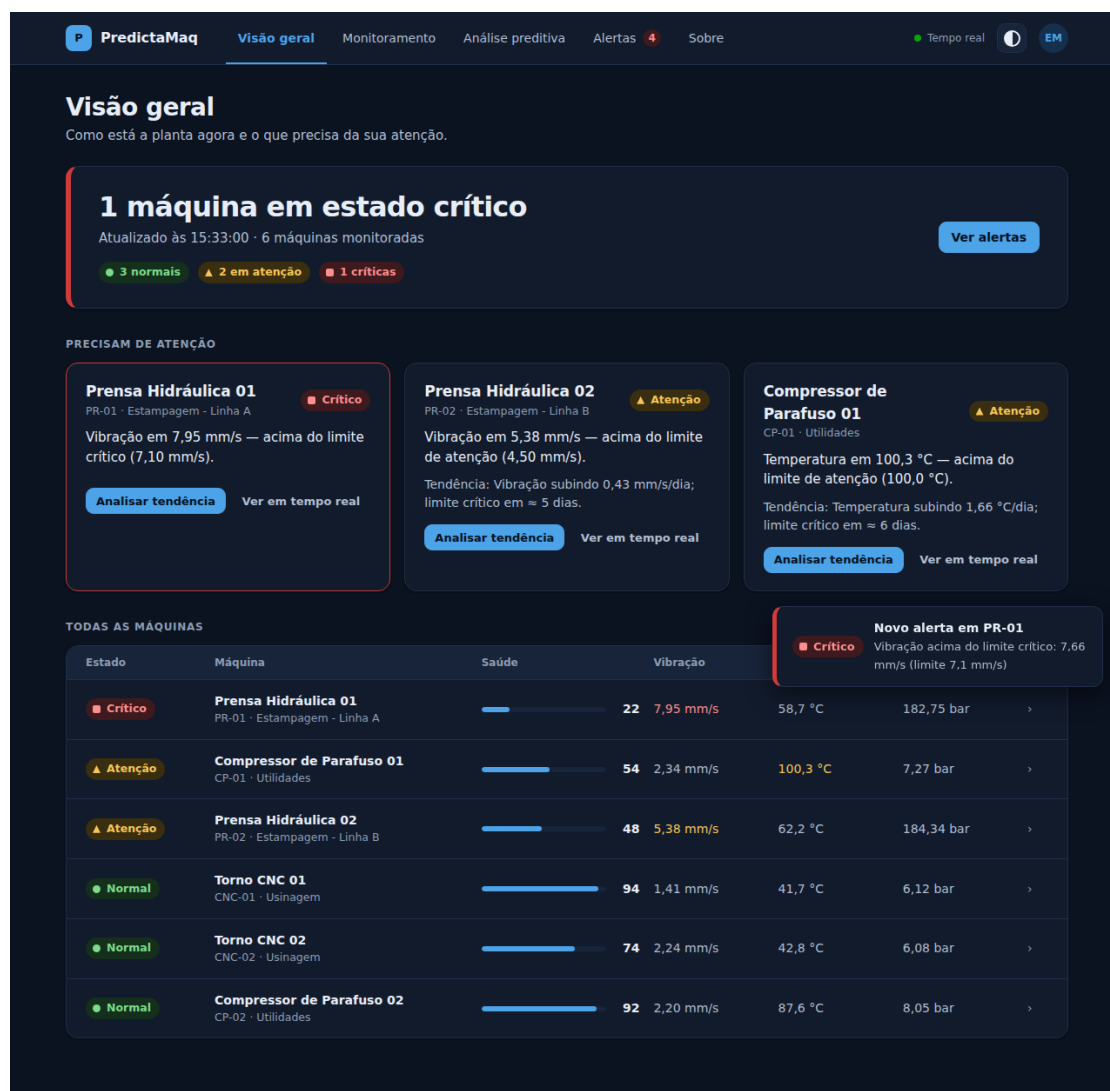


Figura 7: Tema escuro da visão geral.



Figura 8: Página “Sobre”, com abas para princípios de design, diagnóstico crítico, tecnologias, arquitetura e medições.

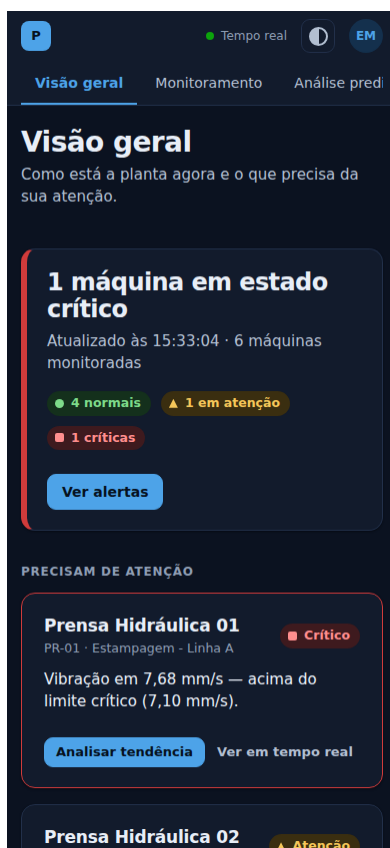


Figura 9: Visão geral em tela de celular (390 px de largura).

7.3 Cores e contraste

A paleta usa o azul #00579D como única cor de destaque (identidade, ações principais e séries dos gráficos), sobre neutros claros ou escuros. Cores de estado (verde, âmbar, vermelho) são reservadas ao significado de estado e sempre acompanhadas de forma e texto. Cada gráfico mostra uma única série, de modo que não há necessidade de paleta categórica. Os contrastes abaixo foram calculados pela fórmula de luminância relativa da WCAG:

Tabela 8: Razões de contraste calculadas (mínimo: 4,5:1 para texto e 3:1 para elementos gráficos).

Par (primeiro plano / fundo)	Tema claro	Tema escuro
Texto principal / superfície	17,3	14,8
Texto secundário / superfície	7,5	9,2
Texto terciário / superfície	5,3	6,3
Azul de destaque / superfície	7,4	6,3
Texto do botão primário / azul	7,4	6,9
Chip “normal” (texto / fundo)	5,9	8,4
Chip “atenção” (texto / fundo)	6,3	8,4
Chip “crítico” (texto / fundo)	6,6	6,9
Indicador verde / superfície	3,4	5,1
Indicador vermelho / superfície	4,8	3,6
Indicador âmbar / superfície	1,8	9,4

A única razão abaixo do mínimo é o indicador âmbar sobre fundo claro (1,8:1). Ela é compensada pela forma (triângulo) e pelo texto (“Atenção”), e as linhas de limite no gráfico claro usam um âmbar mais escuro. O texto terciário no tema claro foi ajustado após a verificação (valor inicial reprovava sobre o fundo da página).

8 Testes e verificação

8.1 Testes unitários

Quatro testes (`pytest`) cobrem: limites de severidade bilaterais da pressão, monotonicidade do índice de saúde, *debounce* do detector de alertas e a estimativa de RUL em uma série linear conhecida (erro menor que 0,5 dia). Todos passam.

8.2 Testes ponta a ponta

Com o Chromium automatizado (Playwright) foram executados: conexão WebSocket (indicador “Tempo real”), troca de variáveis e máquinas, injeção de falha, notificação de novo alerta, reconhecimento do alerta, criação e andamento de ordem de serviço, alternância de tema e capturas de tela em desktop e celular. Não foram escritos testes

automatizados permanentes para a interface; esses roteiros foram executados durante o desenvolvimento.

8.3 Defeitos encontrados e corrigidos

- A pasta `assets` não era carregada (o Dash a procurava dentro do pacote), deixando a página sem estilo. Corrigido apontando a pasta explicitamente.
- As faixas de limite não apareciam nos gráficos com subpainéis, pois o Plotly ignora formas em subpainéis vazios. Corrigido adicionando as séries antes das zonas.
- Alertas duplicados a cada reinício, porque o detector não conhecia condições já existentes. Corrigido inicializando o estado do detector e evitando duplicidade de alertas pendentes do mesmo problema.
- Erro 500 ao exibir a notificação de novo alerta (propriedade inválida num componente de link). Corrigido.
- Mensagem “100,0 acima do limite de 100,0” por arredondamento. Corrigida mostrando uma casa decimal adicional quando necessário.

8.4 Medições de desempenho

Valores medidos durante o desenvolvimento (máquina de desenvolvimento, servidor Flask local), na versão anterior ao redesenho final; devem ser remedidos (a página *Sobre* → *Medições* refaz a medição em tempo real):

Tabela 9: Medições de desempenho do protótipo.

Métrica	Valor
Histórico em CSV	12 096 linhas, 621 KB
Leitura do CSV (pandas, sem cache)	10 ms
Cálculo de estado e índice de saúde (vetorizado)	35 ms
Montagem de uma figura de telemetria (3 painéis, versão anterior)	227 ms
Pacote do WebSocket por tick	1,2 KB
Snapshot completo (equivalente a polling)	28 KB

O pacote por tick é cerca de 23 vezes menor que o snapshot completo, o que ilustra a vantagem do modelo *push* sobre o *polling*.

9 Diagnóstico crítico

Em atendimento à etapa de diagnóstico crítico da proposta, a equipe analisou o próprio protótipo.

9.1 Falhas de usabilidade

- Não há filtro por turno ou linha; o usuário vê todas as máquinas, inclusive as que não são de sua responsabilidade.
- Limites são fixos no código; não podem ser ajustados por máquina na interface.
- O índice de saúde é uma métrica própria e precisa de explicação; há um bloco de detalhes na análise, mas faltam dicas contextuais nos cartões.
- Não existem perfis de usuário, autenticação nem trilha de auditoria nas ordens de serviço.
- A interface não foi validada com usuários reais; as decisões de design se baseiam em princípios e na avaliação da equipe.

9.2 Gargalos de performance e arquitetura

- A cada tick, as figuras são reconstruídas no servidor; não escala para muitas máquinas e usuários simultâneos.
- CSV como banco: leitura completa a cada consulta e regravação integral de alertas e ordens; sem concorrência real entre processos.
- O estado do simulador fica na memória de um único processo; não funciona com vários *workers*.
- O servidor de desenvolvimento do Flask é adequado à demonstração, não à produção.
- O RUL linear é impreciso quando a degradação não é linear ou a série é ruidosa; a confiança baixa é sinalizada, mas o modelo não foi validado com falhas reais.

9.3 Oportunidades de melhoria

- Substituir o CSV por um banco de séries temporais (por exemplo, TimescaleDB ou InfluxDB) e ingerir sensores reais via MQTT ou OPC UA.
- Atualizar o gráfico no navegador apenas com o ponto novo (`extendData`) e aplicar *downsampling* em janelas longas.
- Modelos de degradação mais realistas (exponencial, filtro de Kalman) validados com dados reais.
- Servir com gunicorn e *gevent*, com Redis *pub/sub* para escalar o WebSocket.
- Perfis, autenticação, limites configuráveis e testes automatizados de interface.

10 Atendimento à rubrica

Tabela 10: Relação entre os critérios de avaliação e as evidências do projeto.

Critério	Peso	Evidência
Design e usabilidade	20%	Princípios da Gestalt e hierarquia aplicados (Seção 7); contrastes calculados (Tabela 8); estado por cor, forma e texto; uma pergunta por tela; revelação progressiva.
Programação e frameworks	30%	Dash/Plotly com callbacks reativos, gráficos interativos (hover, projeção, zonas), análises em Python e testes unitários (Seções 2 e 5).
Integração web	20%	WebSocket com reconexão, API REST, layout responsivo, tema claro/escuro, exportação CSV (Seção 6).
Postura crítica e argumentativa	15%	Comparativo de tecnologias, registro do redesenho, defeitos corrigidos e diagnóstico crítico (Seções 3, 8 e 9).
Postura profissional e aprendizado	15%	Processo iterativo com revisão, incorporação de feedback e documentação (README e roteiro).

Entregáveis da proposta. (1) Este relatório técnico; (2) repositório de código organizado em HTML/JS/Python; (3) aplicação web operacional para demonstração em tempo real.

11 Conclusão

O projeto entregou uma aplicação web operacional que cobre o tema específico (manutenção preditiva e telemetria de prensas, compressores e tornos) e os tópicos gerais da proposta: fundamentos de UX, framework de código (Dash/Plotly), integração web (WebSocket e API REST) e uma discussão comparativa com ferramentas de BI. O maior aprendizado do processo foi a necessidade de avaliar criticamente a primeira versão: reduzir a quantidade de informação, organizar a hierarquia e orientar cada tela por uma pergunta do usuário tornou a interface mais clara do que a abordagem inicial de “mostrar tudo”.

As limitações principais são o uso de dados simulados e limites fictícios, o CSV como banco, o redesenho das figuras no servidor e a ausência de validação com usuários reais. Esses pontos foram documentados e orientam os próximos passos.

A Como executar

```

pip install -r requirements.txt
python run.py                # http://localhost:8050
python -m predictive.datagen  # regenera os CSVs do zero
pytest                       # testes unitarios

```

Variáveis de ambiente opcionais: `PORT` (8050), `HOST` (127.0.0.1) e `TICK_SECONDS` (2).

B Roteiro sugerido para a demonstração (15 minutos)

1. Visão geral (2 min): resumo, cartões de atenção, estado por cor, forma e texto.
2. Monitoramento (2 min): abas de variável, zonas de limite, indicador de tempo real, tema escuro.
3. Injeção de falha (2 min): modo demonstração, falha de rolamento na PR-01, notificação e estado crítico.
4. Alertas (2 min): reconhecer, criar ordem, iniciar; ver `data/work_orders.csv` mudar.
5. Análise preditiva (3 min): PR-02, dias até o limite crítico, confiança e limitações.
6. Argumentação (4 min): princípios de design, diagnóstico crítico, tecnologias e melhorias.

Referências

- [1] *Trabalho de Pesquisa e Aplicação Prática: Engenharia e Design de Interfaces Analíticas na Indústria 4.0*. Proposta de trabalho (documento da disciplina).
- [2] FEW, S. *Information Dashboard Design: The Effective Visual Communication of Data*. 2. ed. Analytics Press, 2013.
- [3] TUFTE, E. R. *The Visual Display of Quantitative Information*. 2. ed. Graphics Press, 2001.
- [4] NIELSEN, J. *10 Usability Heuristics for User Interface Design*. Nielsen Norman Group, 1994.
- [5] W3C. *Web Content Accessibility Guidelines (WCAG) 2.x*. World Wide Web Consortium.
- [6] FETTE, I.; MELNIKOV, A. *The WebSocket Protocol*. RFC 6455, IETF, 2011.
- [7] ISO 20816 (série). *Mechanical vibration — Measurement and evaluation of machine vibration*.
- [8] ISO 17359. *Condition monitoring and diagnostics of machines — General guidelines*.
- [9] ISO 13374 (série). *Condition monitoring and diagnostics of machines — Data processing, communication and presentation*.
- [10] PLOTLY. *Dash Documentation*. Disponível em: <https://dash.plotly.com>.
- [11] PLOTLY. *Plotly Python Graphing Library*. Disponível em: <https://plotly.com/python/>.